

PESQUISA APROFUNDADA

PID namespace init em um Docker-like system construído com Python, Cython e C++

Definições, contexto histórico no Docker, análise arquitetural e um plano de implementação orientado por System Design, Low Level Design, Domain Driven Design, padrões GoF e requirements para tratar corretamente reaping, órfãos, repasse de sinais e o exit code do workload.

ESCOPO

Feature “PID namespace init” em um runtime de containers

STACK-ALVO

Control plane em Python, glue em Cython, data plane em C++

DATA

12 de março de 2026

Sumário

0	Resumo executivo	Visão geral
1	Escopo técnico e formulação correta do problema	Fundamentos
2	Conceitos centrais: PID namespace, PID 1, zombies, órfãos, sinais e exit status	Semântica do kernel
3	Como Docker, OCI, containerd e init shims tratam esse tema	Contexto original
4	Arquitetura recomendada para o projeto Python/Cython/C++	System Design
5	Low Level Design: launch, supervisão, signal forwarding, reaping e saída final	Plano detalhado
6	Modelagem de domínio e limites de responsabilidade	DDD
7	Padrões de design aplicáveis	GoF
8	Requirements funcionais e não funcionais	Critérios de aceitação
9	Roadmap incremental, testes e hardening	Execução
10	Conclusão	Síntese
11	Referências	Base documental

0. Resumo executivo

A feature solicitada não é um detalhe periférico do runtime. Ela é um dos pontos onde um sistema “parecido com Docker” deixa de ser apenas um empacotador de processos e passa a se comportar como um runtime de containers correto do ponto de vista do kernel Linux. Em um PID namespace privado, o primeiro processo é o PID 1 e passa a carregar uma semântica especial: ele se torna pai de órfãos, precisa recolher filhos mortos para evitar zombies, só pode receber determinados sinais se tiver handlers instalados e, se morrer cedo demais, o kernel mata o resto do namespace com SIGKILL. Logo, implementar containers sem uma política explícita para o PID 1 costuma produzir exatamente os defeitos clássicos do ecossistema: parada graciosa falha, children ficam zombies, processos em background ficam sem dono e o exit code observado pelo usuário deixa de refletir de forma confiável o workload principal [1][2][3].

TESE CENTRAL

O modo mais sólido de implementar a feature é separar nitidamente o problema em dois planos. No host, um supervisor em C++ usa primitivas modernas do kernel, em especial `clone3`, `CLONE_PIDFD`, `waitid(P_PIDFD)` e `pidfd_send_signal`, para criar e vigiar o processo que será o PID 1 do container. Dentro do namespace, um binário mínimo e dedicado, também em C++, atua como *runtime-init*: ele instala handlers, cria o workload principal, reencaminha sinais segundo uma política explícita, reaplica semântica de stop compatível com a imagem e só encerra depois de drenar todos os descendentes relevantes, preservando o status do workload como resultado oficial do container.

Essa separação é importante porque o controle de ciclo de vida, metadados, API e reconciliação pertence naturalmente ao control plane em Python, enquanto o diálogo com o kernel, os loops de sinais, a interação com descritores e a montagem dos namespaces pertencem ao data plane em C++. O Cython entra como uma fronteira estreita de tipos e ABI, não como o lugar onde se faz `fork`, `clone` ou espera bloqueante em filhos. Em termos práticos, a recomendação desta pesquisa é adotar uma arquitetura parecida com o padrão shim+engine usado por containerd: Python orquestra, um daemon C++ fornece o runtime local e cada container ganha um supervisor próprio ou logicamente isolado, capaz de sobreviver a falhas do processo Python e de reportar status estruturado de volta ao plano de controle [17][18].

Do ponto de vista de produto, vale uma decisão deliberada: um modo “Docker-compatible” e um modo “native-managed”. O primeiro replica a experiência mais fiel do Docker, em que o workload vira PID 1 por padrão e só ganha um tiny init quando o usuário pede algo equivalente a `--init`. O segundo privilegia correção operacional e sempre injeta o *runtime-*

init em namespaces privados. A recomendação para um projeto novo é ter os dois perfis, mas usar o modo gerenciado como default interno para ambientes de plataforma, porque ele reduz defeitos e torna observabilidade, paradas e recuperação muito mais previsíveis.

1. Escopo técnico e formulação correta do problema

O enunciado “respeitar a semântica especial do PID 1; reaping, órfãos, repasse de sinais, exit code do workload” parece, à primeira vista, uma lista de quatro requisitos independentes. Na prática, eles são uma única cadeia causal. Quando um processo roda como PID 1 dentro de um namespace, o kernel altera a maneira como sinais são entregues, passa a reparentar órfãos para esse processo e, se ele encerrar antes do momento correto, o namespace inteiro é abatido. Logo, o reaping não é um detalhe de higiene; é condição para que o namespace não acumule zombies. O repasse de sinais não é só UX; ele define se o container consegue parar graciosamente. E o exit code não é uma convenção cosmética; ele é o contrato que o orquestrador, a API, o CLI e sistemas externos vão consumir para decidir sucesso, falha ou retry.

Também é um erro modelar essa feature como algo localizado apenas “dentro do container”. O problema atravessa três contextos. O primeiro é o contexto do kernel, onde vivem *clone*, *wait*, entrega de sinais, *reparenting*, *subreapers* e *pidfd*. O segundo é o contexto do runtime, que escolhe como cria o processo inicial, como faz a transição de *Create* para *Start*, como detecta falha de *execve* e como expõe um *Wait* confiável. O terceiro é o contexto do control plane, que decide qual semântica expor ao usuário, qual compatibilidade perseguir e quais invariantes persistir em metadados. Se esses três contextos forem acoplados de forma descuidada, o resultado típico é uma mistura ruim: lógica de sinal no Python, espera de filhos em Cython, metadado derivado de status imperfeito e várias condições de corrida difíceis de reproduzir.

ERRO CLÁSSICO DE PROJETO

Uma implementação ingênua costuma assumir que basta criar um processo em um PID namespace e depois enviar *SIGTERM* para ele quando se quer parar o container. Esse desenho falha de pelo menos quatro formas: o processo pode ignorar o sinal por estar como PID 1 sem handler apropriado; descendentes podem sobreviver em background; órfãos podem ser reparentados sem jamais serem recolhidos; e o status de saída que chega ao host pode refletir o wrapper, não o workload. O problema real não é “mandar sinal”; é administrar corretamente uma árvore de processos sob as regras especiais do namespace [1][2][12][13].

Neste documento, a análise será feita com uma suposição operacional importante: o projeto pretende construir um runtime “Docker-like” do zero, não apenas um launcher de processos isolados. Isso implica desenhar contratos explícitos para *Create*, *Start*, *Stop*, *Kill*, *Wait*, *Exec* e *Inspect*, bem como separar desejado e observado. O foco imediato continua sendo o PID

namespace init, mas a solução proposta já nasce encaixada em uma arquitetura de runtime real, capaz de acomodar depois outros aspectos como cgroups, mount namespaces, rootless, tty, seccomp e checkpoint/restore.

2. Conceitos centrais: PID namespace, PID 1, zombies, órfãos, sinais e exit status

2.1 PID namespace e o papel singular do primeiro processo

Um PID namespace isola a numeração de processos e permite que um conjunto de processos enxergue uma hierarquia própria, na qual os PIDs começam em 1 e processos de fora deixam de ser visíveis como pares normais. O primeiro processo criado no namespace passa a ser o “init” daquele espaço e recebe PID 1. O kernel trata esse processo como estrutural: se ele termina, todos os processos restantes do namespace são mortos com SIGKILL e novas criações de processo nesse namespace passam a falhar com ENOMEM [1]. Essa regra existe porque o namespace precisa de um pai de referência para a adoção de órfãos e para o correto fechamento da árvore de processos.

SEMÂNTICA QUE MUDA O DESIGN

Em um sistema host normal, o pai de um processo pode morrer sem que o sistema inteiro entre em colapso. Em um PID namespace privado, a morte do PID 1 tem efeito sistêmico: ela encerra o espaço de processos. Por isso, o “wrapper” que roda como PID 1 não pode ser tratado como detalhe descartável. Ele é parte do contrato de execução.

Há ainda uma nuance frequentemente subestimada: apenas sinais para os quais o init do namespace tenha estabelecido um handler podem ser enviados a ele por outros membros do mesmo namespace. O mesmo vale, com exceções específicas, para sinais vindos de namespaces ancestrais. SIGKILL e SIGSTOP são tratados à parte, mas sinais como SIGTERM, SIGINT e SIGHUP dependem de o PID 1 ter disposição explícita para recebê-los [1][2]. Em outras palavras, um processo que funcionava como serviço comum fora de container pode parar de responder como se espera quando passa a rodar como PID 1.

2.2 Reaping e zombies

Quando um processo filho termina, o kernel mantém um registro mínimo do seu término até que o pai execute `wait()`, `waitpid()` ou `waitid()`. Enquanto isso não acontece, o processo fica em estado zombie. Zombies não consomem CPU como processos normais, mas consomem entradas de tabela de processos e, mais importante, indicam que o contrato pai-filho foi quebrado [3]. Em ambientes de container isso é comum quando o processo principal

lança workers, shells, wrappers ou scripts que fazem fork e nunca recolhem seus descendentes. Se esse processo está no papel de PID 1, o problema se agrava porque ele é também o ponto de adoção de órfãos.

Reaping correto, portanto, significa duas coisas diferentes ao mesmo tempo. A primeira é recolher os filhos que o próprio PID 1 criou, incluindo o workload principal. A segunda é recolher processos que se tornaram seus filhos por reparenting, seja porque um descendente fez double-fork, seja porque um docker exec-like criou uma nova raiz de processo no mesmo namespace e depois abandonou um neto. Uma implementação séria não pode sobrescrever o status final do workload só porque um órfão foi recolhido depois. Ela precisa diferenciar “filho primário” de “qualquer outro processo reaped”.

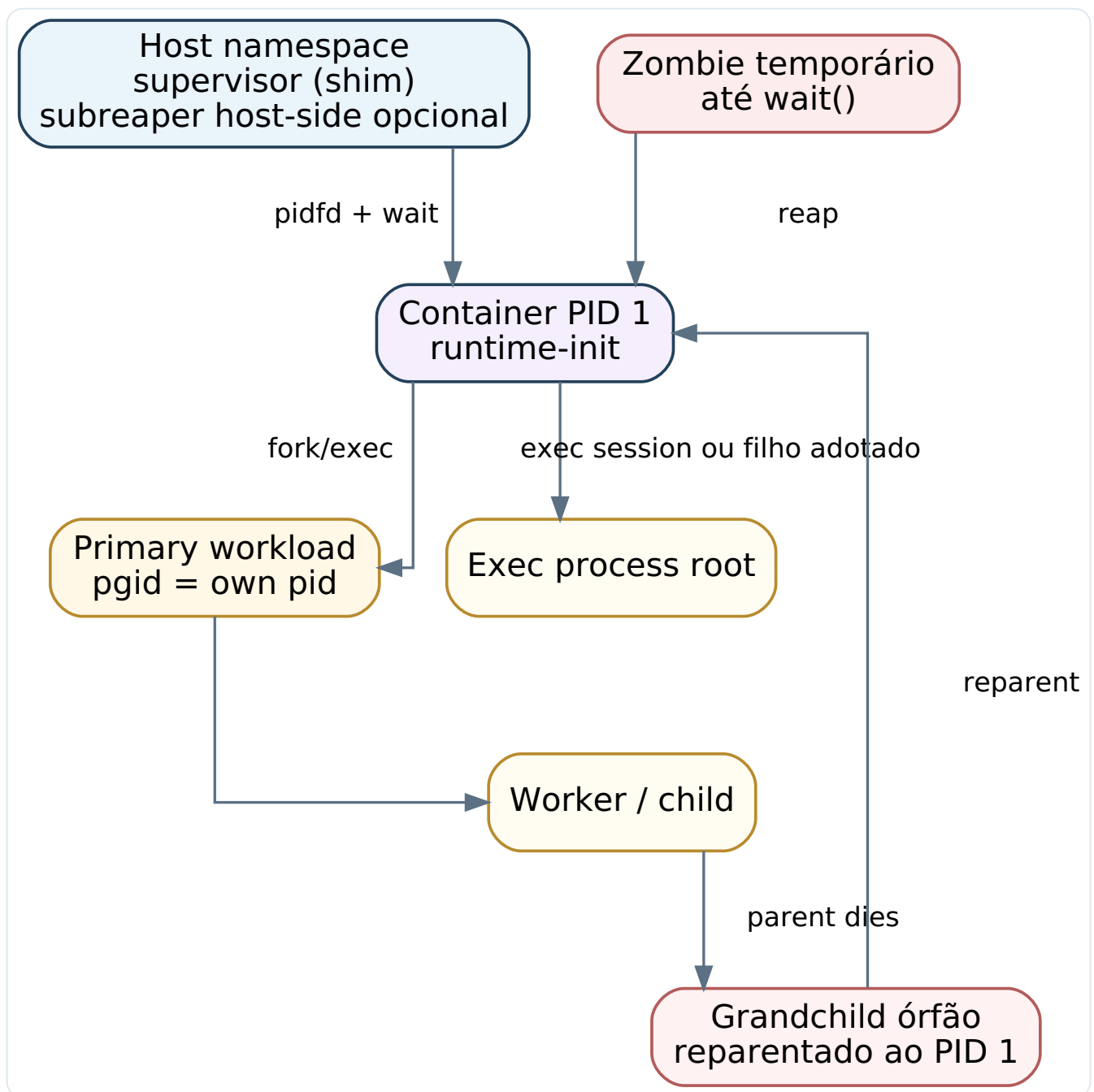


Figura 1. O PID 1 do namespace é o ponto inevitável de adoção e reaping. Um descendente abandonado pode virar filho do init mesmo quando não foi criado pelo workload principal.

2.3 Órfãos e reparenting

Quando um processo fica órfão, ele é reparentado ao init do PID namespace do seu pai ou a um subreaper mais próximo, se houver [1][8]. O detalhe relevante para um runtime de containers é que essa regra interage de forma sutil com `setns()` e execs iniciados a partir do host. O projeto containerd documenta explicitamente que, quando o container roda em um PID namespace novo, o próprio init do container deve limpar órfãos criados por processos de exec; o shim host-side não pode contar com reparenting transfronteira para resolver esse problema [17]. Essa observação é extremamente importante porque ela invalida uma suposição tentadora: a de que o supervisor do host pode ser o reaper universal de tudo. Em namespaces privados, ele não pode.

Para o projeto proposto, a implicação é direta. O domínio “process tree management” precisa reconhecer que o conjunto de PIDs relevantes do container não se reduz ao filho principal. Haverá processos primários, processos de exec, processos adotados e descendentes efêmeros. A política correta é considerar o PID 1 como reaper de último recurso para tudo que vive naquele namespace, mas considerar apenas o workload principal como fonte autoritativa do exit status do container, salvo falha administrativa antes de o workload realmente iniciar.

2.4 Repasse de sinais

Quando um usuário executa um comando equivalente a stop, o sistema não quer “matar um PID”; ele quer pedir ao workload que termine de forma controlada. No Docker, o sinal inicial de parada costuma ser SIGTERM, mas pode ser alterado por STOPSIGNAL na imagem ou por opções de runtime, e se o timeout expira o daemon envia SIGKILL [12][14]. Em sessões acopladas, o Docker também pode fazer proxy de sinais recebidos pelo cliente para dentro do container, comportamento controlado por `--sig-proxy` no attach [13]. O ponto estrutural é que o sinal normalmente chega ao processo que o runtime reconhece como o processo do container, e esse processo precisa decidir se encaminha o sinal ao workload, ao grupo de processos ou a ninguém.

Esse encaminhamento pode ser feito com diferentes granularidades. Encaminhar apenas para o PID do workload é a opção mais compatível com a ideia de “o container é um processo”, mas falha em cenários com shells, wrappers e pipelines. Encaminhar para o process group do workload resolve melhor scripts e aplicações que geram children cooperativos, aproximando-se do comportamento de dumb-init. Encaminhar para o cgroup inteiro é a opção mais agressiva e funciona melhor como escalada de hard stop do que como soft stop. A solução correta, portanto, não é escolher um comportamento único e implícito; é modelar uma *SignalForwardingPolicy* explícita, com defaults diferentes para modo compatível e modo gerenciado [12][15][16].

2.5 Exit status do workload

O status de saída observado pelo usuário precisa responder a uma pergunta precisa: qual foi o destino do workload primário? No Docker, códigos 125, 126 e 127 são usados para certas falhas de infraestrutura ou de execução inicial, enquanto, em condições normais, o exit status do comando dentro do container é propagado para fora [10][11]. Já no nível do kernel,

um processo pode terminar por `exit(code)` ou por sinal, e o `wait` devolve um status bruto que precisa ser interpretado por macros como `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED` e `WTERMSIG` [3]. Misturar esses planos costuma gerar bugs: o runtime registra o código do wrapper, perde a informação de sinal, ou normaliza tudo cedo demais.

MODELAGEM RECOMENDADA

Guarde sempre duas representações. A primeira é estrutural e fiel ao kernel: causa da saída, status bruto de `wait`, sinal, core dump, timestamp e origem da decisão. A segunda é uma projeção para consumo externo: inteiro compatível com CLI, por exemplo `3, 126, 127` ou `128 + signal`. Essa separação impede que a API perca informação e simplifica auditoria, retry e compatibilidade com um frontend “Docker-like”.

3. Como Docker, OCI, containerd e init shims tratam esse tema

3.1 O comportamento padrão do Docker

No Docker clássico, o processo configurado como *ENTRYPOINT/CMD* costuma se tornar o processo principal do container. Se ele estiver em um PID namespace privado, passa a ser o PID 1 daquele espaço. A documentação do Docker observa que, quando `--init` é usado, o daemon insere um processo `init` leve chamado `docker-init`, implementado com base em Tini, para lidar com reaping e signal forwarding [11][15]. Isso já revela um aprendizado histórico do ecossistema: deixar cada aplicação assumir espontaneamente o papel de PID 1 gera problemas reais demais para ser a única opção.

O Docker também formaliza aspectos da borda de produto que são úteis para o projeto em questão. O comando `docker stop` envia um sinal configurável, espera um timeout e depois envia `SIGKILL` [12]. O `attach` usa proxy de sinais por padrão, mas adverte que um processo rodando como PID 1 no container pode ignorar sinais cujo tratamento padrão não esteja implementado pelo aplicativo [13]. E o comando `docker run` reserva certos códigos para falhas do próprio Docker ou da fase de bootstrap, preservando o código do processo do container quando ele de fato iniciou [10][11]. Esses três pontos formam um contrato bastante útil: bootstrap e workload não são a mesma coisa; parada graciosa é tentativa, não garantia; e o runtime precisa distinguir falha de criação de falha do processo do usuário.

3.2 Tini, dumb-init e o aprendizado operacional do ecossistema

Tini descreve seu papel de forma simples: ele cria um único filho, espera sinais e mortes de filhos, reencaminha sinais e reap children, saindo com o código do filho principal [15]. É exatamente a especialização que um `init` mínimo precisa ter. O projeto ainda oferece modo `subreaper` para contextos em que ele não é o PID 1, remapeamento de códigos de saída e

opção de sinalizar o grupo inteiro do filho. Já o *dumb-init* enfatiza o problema da posição especial do PID 1, incluindo a diferença de semântica para sinais e a necessidade de agir como um proxy que, por padrão, cria uma nova sessão e encaminha sinais para todo o process group [16].

Esses dois projetos mostram algo importante para o seu runtime: a feature é pequena em superfície, mas cheia de decisões semânticas. Um init mínimo precisa decidir se encaminha sinais para o processo principal ou para o grupo, se reescreve certos sinais de job control, se se comporta como subreaper fora do PID 1, se remapeia saídas específicas e como lida com parent death signal. Para um projeto novo, copiar um desses programas literalmente pode acelerar um protótipo, mas não substitui a necessidade de modelar a semântica desejada. Como a stack do seu projeto já separa control plane e data plane, faz mais sentido internalizar esses aprendizados e implementá-los como política explícita dentro do runtime.

3.3 containerd-shim e a separação entre host supervision e namespace init

O modelo Runtime v2 do containerd é especialmente instrutivo porque deixa claro que o daemon de alto nível não precisa ser pai direto dos processos do container. O shim existe para manter IO, conservar estado mínimo do task, publicar eventos e coletar status, mesmo se o daemon principal reiniciar [17]. Esse desenho interessa diretamente ao seu projeto. Se o Python controlar tudo de dentro do mesmo processo que faz clone, espera em filhos e gerencia descritores, uma falha ou reinício do control plane vira risco operacional para containers vivos. Se, ao contrário, o Python fala com um daemon ou shim C++ estável, o sistema consegue tolerar falhas do plano de controle sem perder containers em execução.

Há um trecho particularmente relevante no material do containerd: o shim assume responsabilidade como subreaper para limpar containers ou processos setns quando compartilha namespace, mas, quando o container está em um novo PID namespace, o próprio init do container deve limpar processos órfãos reparentados por operações de exec [17]. Esse ponto conecta perfeitamente host supervision e namespace init. O shim do host é excelente para manter pidfd, IO, eventos e status externo. Ele não substitui o init interno como reaper final do namespace.

3.4 OCI Runtime Spec e o contrato de lifecycle

A OCI Runtime Specification descreve operações como *create*, *start*, *kill*, *state* e *delete*, além de um objeto de estado que inclui PID e status do container [18]. Para a feature discutida aqui, a lição mais importante é que o runtime precisa expor um identificador de processo útil para o host, mas isso não elimina a necessidade de distinguir conceitualmente o processo principal do workload. Em um design com *runtime-init*, o PID que o host observa diretamente pode ser o do init wrapper, enquanto o PID conceitualmente “principal” dentro do namespace é o do workload. Os dois são verdadeiros, desde que o modelo os trate como fatos diferentes.

Aspecto	Docker sem --init	Docker com --init	Proposta para o projeto
Quem vira PID 1	O próprio workload.	Tini/docker-init; workload vira filho.	Modo compatível: igual ao Docker. Modo gerenciado: sempre injeta <i>runtime-init</i> .
Reaping de órfãos	Responsabilidade do workload.	Feito pelo tiny init.	Responsabilidade explícita do <i>runtime-init</i> no namespace.
Stop signal	Sinal ao processo principal; se ele é PID 1 sem handler, pode ignorar.	O init recebe e repassa ao filho.	Supervisor host sinaliza PID 1 via <i>pidfd</i> ; PID 1 aplica política de forwarding.
Execs e processos <i>setns</i>	Dependem do comportamento do PID 1 do container.	Melhora com init interno.	Init deve tratar execs e órfãos como parte da árvore do namespace; host shim não basta.
Exit code externo	Do workload, se ele iniciou corretamente.	Normalmente preservado pelo init.	Preservar o status do workload em estrutura rica e projetá-lo para CLI/API.

O saldo dessa comparação é claro. O Docker já ensinou que um runtime real precisa separar, no mínimo conceitualmente, o “processo observado pelo host” do “workload cujo status interessa ao usuário”. O containerd ensinou que esse problema fica mais limpo com um shim independente. E Tini/dumb-init ensinaram que o init interno precisa ser pequeno, específico e sem ambiguidade de responsabilidade. O projeto proposto se beneficia de todos esses aprendizados, mas não precisa ficar preso à compatibilidade absoluta em todas as situações.

4. Arquitetura recomendada para o projeto Python/Cython/C++

4.1 Princípio de separação: Python orquestra, C++ fala com o kernel

A divisão proposta pelo projeto faz sentido técnico: control plane, metadados, API e reconciliação em Python; data plane, abstrações de runtime e interações com o kernel em C++; Cython como fronteira entre os dois. A única condição para esse desenho funcionar bem é não violentar a fronteira. Isso significa que Python não deve ser o lugar onde se faz fork após um processo multithread, não deve manter loops complexos de sinais, não deve

possuir diretamente a semântica de reaping e não deve ser o dono exclusivo de descritores críticos cuja perda mataria containers vivos. Esses são trabalhos de um runtime C++ especializado.

Em vez de uma biblioteca C++ embutida em processo Python com estado vivo de tasks e file descriptors delicados, a recomendação mais robusta é um daemon local C++ com uma API de baixa latência via Unix domain socket. O Cython pode, então, expor um cliente tipado para o Python, com marshaling e liberação de GIL em chamadas bloqueantes. Isso preserva o papel do Cython como “cola” sem transferir para ele a responsabilidade pela vida útil do runtime. O resultado se aproxima da arquitetura shim+engine do containerd, mas adaptado ao seu stack.

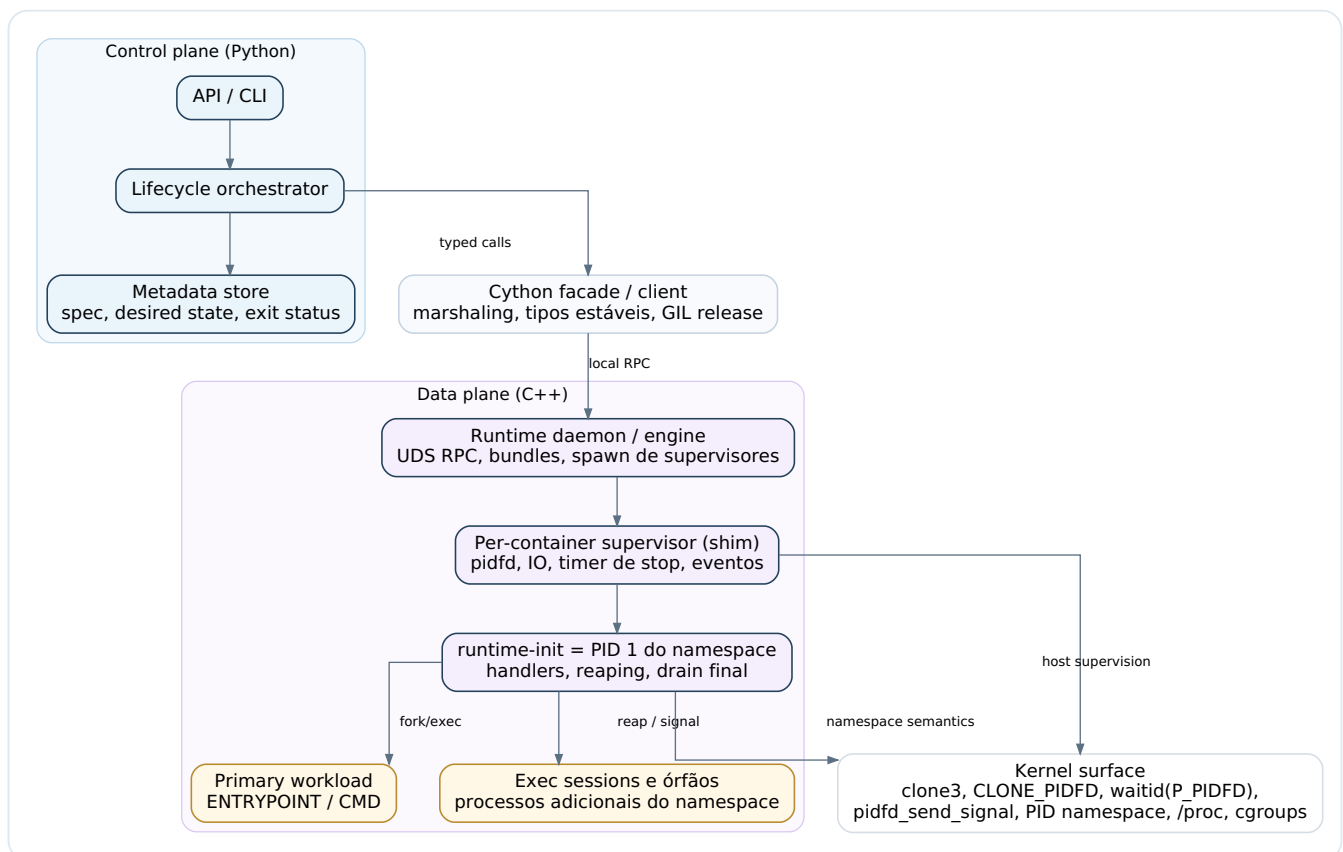


Figura 2. Recomenda-se um boundary fino em Cython sobre um runtime C++ independente. O *runtime-init* interno resolve a semântica do PID 1; o supervisor host usa *pidfd* para lifecycle e observabilidade.

4.2 Componentes propostos

O componente Python precisa concentrar a linguagem de produto do sistema: autenticação, autorização, API externa, scheduler, desired state, reconciliação, persistência de metadados e projeções de status. Ele decide o que o container deveria ser e persiste essa intenção. Já o daemon C++ vive no plano do que o kernel realmente executa: cria bundles, faz `clone3`, administra cgroups, gerencia pipes e TTYs, sobe um supervisor por task e publica eventos factuais. Entre eles, o Cython fornece uma API estável com tipos como `ContainerSpec`, `LaunchOptions`, `SignalPolicy` e `ExitStatus`, sem virar um segundo runtime escondido.

Para cada container, o runtime C++ cria um supervisor dedicado ou uma entidade de isolamento equivalente. Esse supervisor é o pai host-side do processo que se tornará o PID 1 no namespace do container. Ele mantém o `pidfd`, faz `waitid(P_PIDFD)`, cuida de IO, timers de stop e limpeza host-side. O processo que roda como PID 1, por sua vez, é um binário mínimo chamado aqui de *runtime-init*. Seu trabalho é restrito e valioso: instalar handlers, montar `/proc` quando necessário, executar o workload principal, reencaminhar sinais, recolher filhos e só então encerrar o namespace.

DECISÃO ESTRUTURAL RECOMENDADA

Não trate o *runtime-init* como uma biblioteca do workload. Trate-o como um componente formal do runtime. Ele deve ser versionado junto com o engine, ter contrato estável com o supervisor e ser distribuído como um binário controlado pela plataforma, possivelmente injetado no rootfs via bind mount somente leitura, como o Docker faz com seu tiny init [11][15].

4.3 Contratos de API internos

Do ponto de vista de design de serviços, a API entre Python e C++ precisa declarar claramente quando um container foi apenas criado, quando ele iniciou o processo PID 1, quando o workload principal realmente executou com sucesso e quando o status final está fechado. Isso evita uma classe comum de bugs em que *Start* retorna sucesso antes do `execve` do workload, levando a estados fantasmas. Em termos práticos, a operação *Start* deveria retornar somente após um handshake do *runtime-init* indicando que o workload principal fez `execve` com sucesso ou, em caso de falha, retornando um erro estruturado com o erro correspondente.

Outra decisão relevante é a forma como o sistema expõe PIDs. A API de inspeção deve diferenciar pelo menos três campos: o PID host do *runtime-init*, que é útil para supervisão via `pidfd`; o PID do workload principal dentro do namespace do container; e, opcionalmente, o PID host correspondente ao workload, quando isso for útil para debugging. Colapsar tudo em um único campo `pid` leva a confusão. O OCI runtime state já pressupõe que há um PID relevante para o host [18], mas o seu modelo de domínio pode ser mais explícito do que o mínimo da OCI.

4.4 Compatibilidade versus segurança operacional

Uma plataforma nova não precisa herdar cegamente todos os defaults históricos do Docker. O comportamento padrão do Docker faz o workload virar PID 1, o que é compatível, mas também transfere para a aplicação a responsabilidade por reaping e signal semantics. Em uma plataforma interna, esse default raramente é o mais seguro. Por isso, a arquitetura sugerida admite dois perfis. No perfil compatível, o init gerenciado só é injetado quando o usuário pede algo equivalente a `--init`. No perfil gerenciado, o runtime sempre põe um

runtime-init entre o namespace e o workload, preservando o exit status do workload e reportando explicitamente essa mediação. Essa flexibilidade permite atender tanto casos de compatibilidade quanto casos de operação confiável.

5. Low Level Design: launch, supervisão, signal forwarding, reaping e saída final

5.1 Lançamento do container: por que `clone3` e `pidfd` devem ser o caminho padrão

No host, o supervisor do container deve criar o processo inicial usando `clone3` com pelo menos `CLONE_NEWPID` quando a política pedir PID namespace privado e, idealmente, `CLONE_PIDFD` para obter de forma atômica um descritor estável para esse processo [4][5]. A vantagem operacional do `pidfd` é simples e enorme: ele elimina a classe de corrida em que um PID numérico é reciclado e um sinal ou `wait` acerta o processo errado. Para um runtime, isso significa *Stop* e *Wait* muito mais confiáveis, principalmente sob falhas, reinícios e alta concorrência [5][6].

Também vale usar `CLONE_CLEAR_SIGHAND` quando disponível, ou uma rotina equivalente de reset explícito de disposições de sinal no child, para garantir que o processo que se tornará o PID 1 não herde handlers acidentais do ambiente do supervisor [4]. Essa cautela é especialmente importante se o supervisor for um processo longo que já manipula sinais, timers, profiling ou tracing. O *runtime-init* deve começar sua vida com uma tabela de sinais limpa, previsível e definida pelo runtime, não pelo acaso da infraestrutura.

Se o launch incluir também novo mount namespace, o processo que está prestes a virar PID 1 deve montar uma nova instância de `/proc` dentro desse namespace. Sem isso, ferramentas como `ps` e introspecção de PIDs passam a enxergar uma visão inconsistente, porque o conteúdo de `/proc` depende do namespace do processo que montou o filesystem [1]. Essa operação deve acontecer cedo no bootstrap do *runtime-init*, depois de entrar nos namespaces relevantes e antes do workload ganhar controle.

5.2 O binário *runtime-init* dentro do namespace

O *runtime-init* deve ser um executável pequeno, monolítico e muito previsível. Em vez de tentar ser um mini-shell ou um supervisor genérico, ele deve fazer apenas o que o PID 1 precisa fazer: instalar handlers, criar o workload principal, coletar mortes de children, encaminhar sinais e drenar a árvore remanescente antes de encerrar. Como ele roda em uma posição extremamente sensível, C++ aqui deve ser usado com disciplina de sistema: RAII para descritores, nenhuma alocação extravagante em handlers, logging defensivo e uma superfície mínima de dependências.

Há uma questão sutil sobre o mecanismo de ingestão de sinais. Em processos comuns, usar `signalfd` é atraente porque integra sinais ao loop de descritores. No PID 1 do namespace, porém, a regra do kernel fala em sinais para os quais o init estabeleceu um handler [1][7]. Por isso, a opção mais conservadora e segura é instalar `sigaction` explicitamente para os

sinais que o runtime pretende aceitar e usar um self-pipe ou flags atômicas para acordar o loop principal. `signalfd` ainda pode ser usado no supervisor host-side, onde a semântica do PID 1 não existe, ou como otimização adicional depois de a disposição do sinal já ter sido tornada explícita.

SINAIS MÍNIMOS QUE MERECEM HANDLER EXPLÍCITO

Para um runtime de containers, é prudente instalar handlers ao menos para SIGTERM, SIGINT, SIGHUP, SIGQUIT, SIGUSR1, SIGUSR2 e SIGCHLD. SIGKILL e SIGSTOP não podem ser capturados. O conjunto exato pode variar, mas a lógica central é: o PID 1 deve ter disposição explícita para os sinais de operação que o runtime ou o usuário pode enviar [1][2].

5.3 Handshake de bootstrap: só declare “started” depois do execve do workload

Um bom runtime precisa diferenciar “o PID 1 nasceu” de “o workload principal iniciou de fato”. A maneira clássica de fazer isso é um pipe de erro de exec. O *runtime-init* cria um pipe antes de fazer fork do workload. No child, a ponta de escrita é marcada com CLOEXEC. Se o `execve` funcionar, o descritor some automaticamente; se falhar, o child escreve o `errno` e sai. O *runtime-init* lê esse pipe, sabe imediatamente se houve falha de bootstrap e consegue mapear `ENOENT` para 127, `EACCES/ENOEXEC/EPERM` para 126 e outros erros para uma categoria de falha de runtime ou `create/start`. Só depois desse handshake o supervisor deve publicar que o container entrou em *running* [10][11].

```
// Pseudocódigo simplificado do bootstrap dentro do runtime-init
install_signal_handlers();
mount_proc_if_needed();

create_exec_error_pipe();
pid_t child = fork();

if (child == 0) {
    setpgid(0, 0); // workload vira líder do próprio grupo
    execve(argv[0], argv, envp); // se falhar, escreve errno e sai
    write(exec_err_fd, &errno, sizeof(errno));
    _exit(map_exec_errno_to_code(errno));
}

primary_pid = child;
primary_pgid = child;
if (read(exec_err_fd, &err, sizeof(err)) > 0) {
    record_preexec_failure(err);
    drain_and_exit();
}
publish_primary_started(primary_pid);
```

Esse handshake é essencial para o control plane. Sem ele, o Python corre o risco de persistir um estado “running” para um container cujo binário nunca executou. Depois, quando um `Wait` volta 127, fica difícil distinguir “comando não encontrado” de “o processo subiu e falhou”. O custo do pipe é baixo; o ganho de consistência é enorme.

5.4 Política de signal forwarding

A política de sinais deve ser materializada como um objeto de configuração explícito, idealmente imutável após o container iniciar. Esse objeto precisa carregar pelo menos três decisões: qual é o sinal de soft stop efetivo; qual o escopo do forwarding; e qual o timeout de escalada. O sinal efetivo resulta da ordem de precedência mais intuitiva para o usuário: override explícito da chamada de stop, depois configuração do container, depois `STOPSIGNAL` da imagem e, por fim, default `SIGTERM` [12][14]. O escopo do forwarding pode ser *Process*, *ProcessGroup* ou, em camadas mais agressivas, *Cgroup*. O timeout governa quando o supervisor sai do modo gracioso e entra no hard kill.

Minha recomendação é que o perfil compatível use *Process* como default, porque preserva mais de perto a ideia do Docker de que o processo do container é o alvo primário. Já o perfil gerenciado deve preferir *ProcessGroup*, porque é o comportamento que melhor cobre shells, scripts, wrappers e pipelines sem exigir disciplina perfeita do aplicativo. A experiência do *dumb-init* mostra por que isso importa: se um shell recebe `SIGTERM` mas seus children não, o container pode ficar preso em um shutdown incompleto [16]. Em contrapartida, sinais de job control exigem cuidado se houver nova sessão; podem demandar tratamento especial ou versão 2 da feature.

No host, a entrega do sinal ao PID 1 do container deve usar `pidfd_send_signal` sempre que possível [6]. Isso resolve ambiguidades de PID reutilizado e evita depender de buscas frágeis por número de processo. Dentro do namespace, o *runtime-init* pode então repassar o sinal usando `kill(primary_pid, sig)` ou `kill(-primary_pgid, sig)`, isto é, APIs nativas do mesmo namespace em que o workload vive. Essa divisão é elegante: o host usa `pidfd` porque supervisiona por identidade estável; o PID 1 usa PIDs e process groups locais porque opera na semântica interna do namespace.

5.5 Reaping correto e preservação do status do workload

O loop principal do *runtime-init* precisa tratar `SIGCHLD` como um evento de drenagem, não como um “child único morreu”. Em cada notificação, ele deve executar um loop de `waitpid(-1, &status, WNOHANG)` ou equivalente com `waitid` até que não reste mais filho pronto para coleta [3]. Para cada PID reaped, o runtime consulta sua tabela interna. Se o PID corresponde ao workload principal e seu status ainda não foi registrado, esse status se torna a base do resultado final do container. Se o PID corresponde a um exec conhecido, esse resultado vai para a projeção da sessão de exec. Se o PID é desconhecido, ele é tratado como órfão ou descendente adotado: deve ser logado e contabilizado, mas não pode sobrescrever o destino do workload principal.

Esse ponto merece ênfase porque é onde implementações caseiras se perdem. Um container com um shell wrapper, workers, double-forks e exec sessions inevitavelmente produzirá mortes em ordem não determinística. Se o runtime assumir que “o último child reaped é o

status do container”, errará. Se assumir que “o primeiro child que morrer é o container”, também errará em casos de wrappers. O único critério estável é ter uma identidade autoritativa para o *primary workload* desde o momento do fork inicial e tratar todos os demais PIDs como auxiliares ou independentes em relação ao resultado final do container.

5.6 O momento mais crítico: quando o workload principal termina

Quando o workload principal termina, o *runtime-init* não deve simplesmente chamar `exit()` com o mesmo código. Se ele fizer isso e ainda houver descendentes vivos, o kernel matará o resto do namespace com SIGKILL [1]. Às vezes isso parece conveniente, mas é uma má política por duas razões. Primeiro, porque ela torna o shutdown menos gracioso do que poderia ser. Segundo, porque mistura o significado da saída do workload com a limpeza administrativa dos remanescentes. O comportamento mais sólido é o seguinte: registrar o status final do workload principal, entrar em modo de drenagem, sinalizar sobras conforme a política de encerramento e só então encerrar o PID 1 quando não houver mais children ou quando uma escalada administrativa tiver sido necessária.

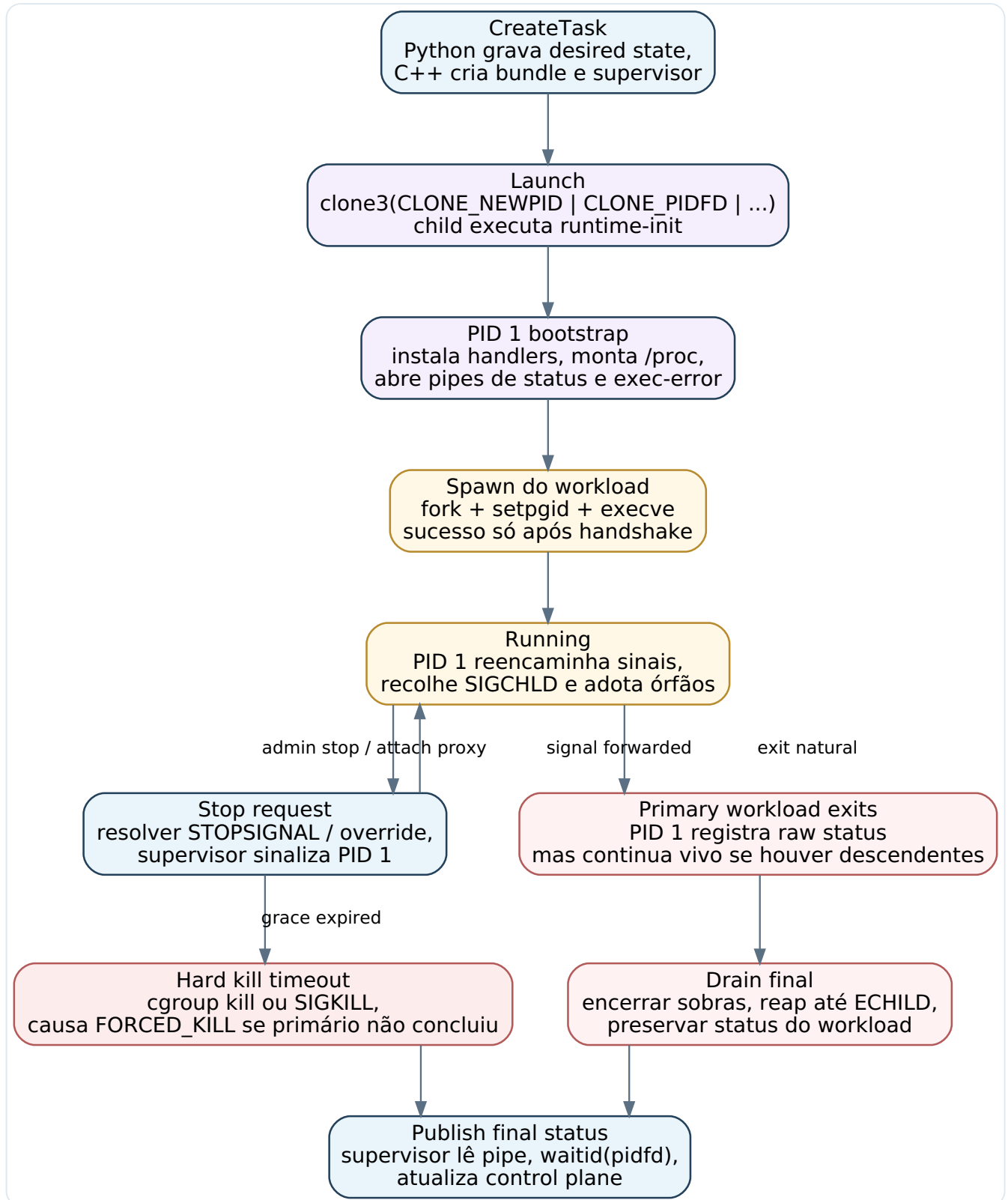


Figura 3. O *runtime-init* não deve sair imediatamente quando o workload termina. Ele precisa drenar a árvore restante para evitar que a semântica do PID 1 destrua o namespace prematuramente.

Isso implica manter separado o “status do workload” e o “status administrativo da limpeza”. Se o workload saiu com código 0, mas um órfão precisou ser abatido com SIGKILL na drenagem final, a saída oficial do container ainda deve ser 0, acompanhada de um campo de observabilidade que registre a limpeza forçada. Se, por outro lado, o workload ainda não havia terminado e o runtime precisou hard-kill o namespace por timeout, então o resultado

oficial deve indicar uma causa *FORCED_KILL* ou, no mínimo, uma normalização compatível como 137, preservando internamente que a causa foi administrativa, não uma escolha do aplicativo. Essa diferenciação melhora muito a interpretação operacional de falhas.

5.7 Canal de status estruturado entre *runtime-init* e supervisor

Embora o *runtime-init* possa sair com o mesmo inteiro do workload principal para compatibilidade com frontends simples, o supervisor host-side não deveria depender apenas do código de saída do wrapper. O ideal é haver um pipe ou socket de status, escrito pelo *runtime-init* antes do encerramento final, com uma estrutura contendo: identificação do primary workload, status bruto de wait, causa normalizada, sinal, tempo de início e fim, indicação de pre-exec failure, quantidade de órfãos reaped e flags de hard kill. Isso transforma o *runtime-init* em um emissor de eventos sem precisar inflá-lo com semântica de API.

```
struct PrimaryExitStatus {
    uint32_t version;
    pid_t primary_pid_inside_ns;
    int raw_wait_status;
    int normalized_exit_code;
    int signal_number;
    bool exited_normally;
    bool exited_by_signal;
    bool core_dumped;
    bool preexec_failure;
    bool forced_kill;
    uint32_t adopted_children_reaped;
    uint64_t started_at_ns;
    uint64_t finished_at_ns;
};
```

Com esse canal, o supervisor combina duas fontes. O pidfd diz quando o PID 1 terminou e oferece identidade estável para *Stop* e *Wait*. O pipe de status diz o que aconteceu de fato com o workload. Essa divisão reduz ambiguidade e torna o comportamento observável sem inflar a API com inferências frágeis.

5.8 Supervisor host-side: responsabilidades e limites

O supervisor host-side deve ser propositalmente simples: dono do pidfd do PID 1, dos descritores de IO, do timer de stop e da comunicação com o daemon C++. Ele não deveria tentar “interpretar” a árvore interna do container além do necessário. Sua função é aplicar lifecycle do lado de fora: criar, iniciar, parar, aguardar, publicar eventos e limpar recursos host-side. Se ele precisar atuar como subreaper no host para certos helpers ou processos em namespaces compartilhados, isso é razoável e o `PR_SET_CHILD_SUBREAPER` existe exatamente para esse tipo de papel [8]. O que ele não deve fazer é acreditar que esse subreaping do host substitui o reaping do PID 1 dentro de namespaces privados, porque não substitui [17].

Outro detalhe útil é usar `PR_SET_PDEATHSIG` em helpers efêmeros criados pelo supervisor durante bootstrap, para evitar processos de preparação esquecidos caso o supervisor morra em momento ruim [9]. Eu não aplicaria isso cegamente ao *runtime-init* principal, porque a estratégia desejável para um runtime moderno é que o container continue rodando mesmo se o processo Python cair. A resiliência deve existir entre control plane e runtime; a morte do supervisor do container, em si, ainda é um evento grave que a arquitetura deve minimizar por isolamento e não por “acoplamento suicida” ao pai.

5.9 Integração com exec e processos adicionais no mesmo namespace

Uma feature exec-like complica o cenário porque introduz novas raízes de processo dentro do mesmo container. O supervisor host-side pode criar uma sessão de exec via `setns` e `execve`, mas, uma vez lá dentro, descendentes orfanizados passam a obedecer às regras do namespace. O material do containerd é explícito: quando o PID namespace do container é diferente do do shim, o init do container deve limpar órfãos reparentados criados por processos de exec [17]. Portanto, o *runtime-init* precisa aceitar que surgirão children que não são o workload principal e que podem morrer muito depois dele.

Do ponto de vista de design, a melhor solução é registrar exec sessions como entidades próprias, com seus próprios IDs e status, mas sem deixar que elas concorram com o *primary workload* pela posse do status final do container. O *runtime-init* não precisa conhecer todos os detalhes da sessão de exec, mas precisa distinguir ao menos entre o PID primário do container e “outros PIDs rastreáveis”. Isso pode ser feito por uma pequena tabela sincronizada pelo supervisor ou, na versão inicial, por uma política ainda mais simples: tudo que não for o primary workload é secundário e jamais muda o resultado oficial do container.

5.10 Interface Cython: fina, tipada e sem ownership accidental

No boundary Python-C++, o Cython deve expor uma API pequena e estável. Em vez de vazar PIDs numéricos, file descriptors ou ponteiros nativos diretamente para o Python, prefira handles opacos e objetos de valor serializáveis. Chamadas bloqueantes como `wait()`, `events()` e operações de streaming devem soltar o GIL. Exceções nativas do C++ precisam ser convertidas em erros Python ricos, mas não devem alterar o estado do runtime por conta própria. O ownership de recursos do kernel permanece no daemon/supervisor C++; o Python recebe projeções, não chaves físicas do motor.

Essa decisão facilita evolução futura. Se mais tarde você trocar o transporte entre Python e C++ de binding direto para RPC local, ou introduzir uma camada de supervisor persistente com reconexão, o Python quase não muda. É exatamente a função correta do Cython nesse projeto: estabilizar o contrato de linguagem, não assumir a carga do runtime.

6. Modelagem de domínio e limites de responsabilidade

6.1 Bounded contexts

Do ponto de vista de Domain Driven Design, a feature se torna muito mais legível quando separada em alguns bounded contexts. Há um contexto de *Container Lifecycle Orchestration*, dono da linguagem de produto de criar, iniciar, parar, aguardar e remover. Há um contexto de *Runtime Execution*, dono da realidade do kernel, do bundle e dos descritores. Há um contexto de *Process Tree Management*, que trata explicitamente primary workload, exec sessions, órfãos e reaping. E há um contexto de *Observability and Audit*, que transforma fatos brutos de runtime em eventos, logs, métricas e histórico legível. O grande ganho dessa divisão é impedir que “status de container” vire uma palavra única que tenta representar tudo.

No control plane Python, o agregado principal é o *Container*. Ele guarda identidade, spec desejada, política de sinais, imagem resolvida, modo de compatibilidade, estado desejado, estado observado e resultado final. Já no data plane C++, o objeto central é o *RuntimeTask* ou *TaskInstance*, que conhece bundle, PIDs, pidfds, cgroup e descritores. O *runtime-init* em si pode ser modelado no domínio como uma entidade técnica chamada *NamespaceInit*, com invariantes próprios, ainda que sua implementação concreta seja um processo externo. Essa modelagem ajuda a evitar um erro sutil: pensar que o init é invisível só porque o usuário não o pediu explicitamente.

6.2 Entidades, value objects e invariantes

Uma modelagem simples e eficaz incluiria as entidades *Container*, *PrimaryWorkload* e *ExecSession*. O *Container* é a raiz de agregado e dono do resultado oficial. O *PrimaryWorkload* é a entidade cujo destino define o exit status do container. Cada *ExecSession* possui ciclo de vida próprio e nunca pode usurpar esse papel. Como value objects, vale ter *SignalPolicy*, *ExitStatus*, *NamespacePolicy* e *CompatibilityProfile*. Esses objetos são bons candidatos porque descrevem configuração e fatos derivados, não identidade mutável.

As invariantes mais importantes são poucas e poderosas. Em um container com PID namespace privado, deve haver no máximo um *NamespaceInit* autoritativo. Deve haver exatamente um *PrimaryWorkload* por container. Um evento de reaping de órfão jamais pode substituir um *PrimaryExitStatus* já registrado. Um container só entra em estado final depois que o runtime observou o encerramento do PID 1 e consolidou o status do primary workload. E uma sessão de exec nunca deve alterar a política de parada do container principal. Se essas invariantes forem codificadas logo no modelo, várias classes de bug deixam de existir.

6.3 Linguagem ubíqua sugerida

Uma boa linguagem ubíqua para a equipe evita o uso solto da palavra “processo”. Convém padronizar termos como *runtime init*, *primary workload*, *adopted child*, *exec session*, *soft stop*, *hard stop*, *forced kill*, *observed state* e *finalized exit status*. Quando todos chamam o mesmo conceito pelo mesmo nome, o design fica menos ambíguo e as APIs ficam melhores. Por exemplo, uma operação `Container.wait()` deveria devolver um `FinalizedExitStatus`, não apenas um inteiro chamado `code`. Já um evento do supervisor poderia ser `AdoptedChildReaped`, e não “child exited”, porque isso explicita que o evento não altera o destino do container.

Essa disciplina de linguagem também melhora a integração Python/C++. O contrato de Cython pode refletir exatamente esses tipos e estados, tornando a fronteira autoexplicativa. O benefício não é acadêmico: quando as palavras são boas, a chance de implementar semântica errada diminui.

7. Padrões de design aplicáveis

Aqui vale ser seletivo. Não é útil “colar” todos os padrões GoF no runtime. Alguns, porém, encaixam de forma natural e aumentam muito a clareza do desenho.

Padrão	Aplicação sugerida	Problema que resolve
State	Modelar o lifecycle do container e do primary workload: <i>Created</i> , <i>Starting</i> , <i>Running</i> , <i>Stopping</i> , <i>Draining</i> , <i>Stopped</i> , <i>Failed</i> .	Evita transições inválidas, simplifica idempotência de <i>Stop/Kill/Delete</i> e centraliza regras por fase.
Strategy	<i>SignalForwardingStrategy</i> , <i>StopSignalResolutionStrategy</i> e <i>ExitNormalizationStrategy</i> .	Permite variar compatibilidade Docker, modo gerenciado, alvo de sinal e mapeamento de status sem espalhar condicionais.
Command	<i>CreateTaskCommand</i> , <i>StartTaskCommand</i> , <i>StopTaskCommand</i> , <i>ExecCommand</i> , <i>WaitCommand</i> .	Dá forma transacional às operações do runtime e facilita fila, auditoria, retry e replay.
Observer	Fluxo de eventos do supervisor para o daemon e do daemon para o control plane.	Separa execução do kernel de projeções de estado, logs, métricas e notificações externas.
Facade / Adapter	Cython como facade tipada para o cliente Python e adapter para a API nativa do daemon C++.	Esconde detalhes de ABI, marshaling e transporte; mantém a API Python estável.

Padrão	Aplicação sugerida	Problema que resolve
Builder	Construção imutável de <i>LaunchSpec</i> , <i>NamespaceSpec</i> e <i>SignalPolicy</i> .	Evita specs parcialmente preenchidas e reduz erro humano na composição de runtime config.

O padrão *State* é particularmente valioso aqui. Quase toda ambiguidade operacional do runtime nasce de transições mal definidas: um stop chega enquanto o container ainda não terminou o execve; um wait é chamado antes de haver status final; um delete é tentado enquanto o PID 1 ainda está drenando órfãos. Se cada fase do lifecycle possuir comportamento próprio e transições explícitas, fica muito mais fácil manter o runtime determinístico.

Strategy também merece destaque porque a discussão de sinais não tem um único comportamento correto. Há um comportamento compatível com Docker, um comportamento mais seguro para scripts e um comportamento agressivo de hard stop. Tornar isso uma estratégia configurável é melhor do que enterrar as decisões em branches de if espalhados pelo código. Da mesma forma, normalização de exit status deve ser estratégia porque o kernel, a API interna e o CLI externo têm necessidades diferentes.

Já o *Observer* é o que fecha o desenho distribuído. O supervisor emite fatos; o daemon os consome e consolida; o Python projeta em metadados e telemetria. Essa é uma maneira limpa de preservar isolamento sem perder visibilidade. Em vez de o Python “adivinhar” o que aconteceu por polling imperfeito, ele recebe eventos inequívocos de start, forwarded signal, adopted child reaped, primary exited, hard kill escalated e finalized status.

8. Requirements funcionais e não funcionais

8.1 Requirements funcionais

ID	Requirement	Descrição e observação
FR-01	Criação correta do PID 1	O runtime deve criar explicitamente o processo init do namespace quando o perfil de compatibilidade ou de gestão assim o exigir, garantindo que o primeiro processo do namespace seja controlado pelo engine.
FR-02	Handlers explícitos no PID 1	O <i>runtime-init</i> deve instalar handlers para o conjunto de sinais operacionais suportados antes de anunciar readiness, respeitando a semântica especial do PID 1 [1][2].
FR-03	Reaping completo	

ID	Requirement	Descrição e observação
		O PID 1 deve recolher filhos diretos, exec sessions e órfãos reparentados, sem deixar zombies persistentes após o encerramento do container.
FR-04	Forwarding configurável	O runtime deve suportar políticas de forwarding para o processo primário e para o process group, com possibilidade de escalada administrativa posterior.
FR-05	Stop signal efetivo	O sinal de soft stop deve ser resolvido por precedência entre override de operação, configuração do container, imagem e default do runtime [12][14].
FR-06	Handshake de start	O sistema só pode publicar que o container iniciou após o execve bem-sucedido do workload primário ou erro estruturado de pre-exec.
FR-07	Status final do workload	O resultado oficial do container deve refletir o destino do primary workload, não o de wrappers, órfãos ou exec sessions secundárias, exceto em falhas administrativas antes de o workload concluir.
FR-08	Estrutura rica de saída	A API interna deve expor causa, status bruto, sinal, código normalizado, timestamps e flags de hard kill, além de projeção inteira para CLI.
FR-09	Suporte a exec	O runtime deve permitir sessões de exec sem comprometer a invariância do primary workload como dono do status final do container.
FR-10	Drain após saída do primário	Quando o workload primário terminar, o PID 1 deve continuar ativo até drenar ou eliminar processos remanescentes, evitando término prematuro do namespace [1].
FR-11	Wait confiável	O host supervisor deve fornecer espera confiável para término via pidfd ou mecanismo equivalente, sem corridas de PID reutilizado [5][6].
FR-12	Observabilidade de orfandade	Eventos de órfãos adotados e reapados devem ser publicados para logs, métricas e debug, ainda que não alterem o resultado final do container.
FR-13	Compatibilidade opcional	O runtime deve oferecer um perfil compatível com o comportamento padrão do Docker e um perfil gerenciado voltado à correção operacional.

ID	Requirement	Descrição e observação
FR-14	Exposição explícita de identidades	<i>Inspect</i> deve distinguir PID host do init, PID do workload dentro do namespace e, quando possível, outras identidades úteis para troubleshooting.
FR-15	Montagem correta de /proc	Em cenários com novo PID namespace e mount namespace, o runtime deve montar uma nova instância de /proc para introspecção consistente [1].

8.2 Requirements não funcionais

ID	Requirement	Descrição e observação
NFR-01	Robustez a falhas do control plane	Containers em execução devem sobreviver a falhas ou reinícios do processo Python. O runtime C++ e seus supervisores precisam ser capazes de continuar operando e de se reconectar posteriormente.
NFR-02	Ausência de corridas por PID	Operações de wait e kill do host devem usar pidfd ou mecanismo equivalente que impeça confusão com PIDs reciclados [5][6].
NFR-03	Previsibilidade de shutdown	O tempo de parada graciosa e o ponto de escalada para hard kill precisam ser determinísticos e auditáveis.
NFR-04	Zero zombie leak na saída	Após o término de um container, não devem restar zombies associados ao namespace do task ou a helpers do bootstrap.
NFR-05	Observabilidade de primeira classe	Eventos, métricas e logs devem permitir reconstruir a história do task: start, signal forwarding, exits, órfãos, escaladas e finalização.
NFR-06	Segurança do boundary	Cython deve ser fino e sem ownership acidental; o uso de file descriptors e handles do kernel deve permanecer no C++.
NFR-07	Baixo overhead	O <i>runtime-init</i> deve ser mínimo em CPU e memória, sem custo relevante frente ao workload principal.
NFR-08	Compatibilidade progressiva com kernels	O runtime deve detectar disponibilidade de clone3, pidfd e recursos correlatos, fazendo feature negotiation quando necessário.
NFR-09	Testabilidade	

ID	Requirement	Descrição e observação
		Todos os cenários centrais de sinais, órfãos, exec, timeouts e códigos de saída devem ser reproduzíveis por testes automatizados de integração.
NFR-10	Manutenibilidade	O código do runtime deve permanecer pequeno, com responsabilidades explícitas e cobertura adequada, evitando espalhar semântica de processo em várias camadas.
NFR-11	Portabilidade prática	O design deve ser viável em x86_64 e arm64 Linux contemporâneos, ainda que certas otimizações dependam de versões recentes do kernel.
NFR-12	Auditabilidade do resultado	A API precisa preservar informação suficiente para explicar por que um container terminou com determinado status, inclusive distinguindo sinalização administrativa de falha natural.

9. Roadmap incremental, testes e hardening

9.1 Fases de implementação recomendadas

A primeira fase deve entregar a espinha dorsal correta, não todos os detalhes de compatibilidade. O objetivo inicial é ter supervisor host-side em C++, launch por `clone3`, `pidfd`, um *runtime-init* mínimo e status estruturado do primary workload. Nessa fase, vale suportar apenas soft stop por SIGTERM e forwarding para o process group. O ganho já é enorme: o runtime deixa de depender do processo do usuário para se comportar como PID 1 e passa a ter um *Wait* confiável.

A segunda fase deve trazer as políticas explícitas: resolução de stop signal por precedência, perfil compatível versus gerenciado, timeout de escalada, diferenciação entre *Process* e *ProcessGroup* e status normalizado compatível com CLI. A terceira fase é onde entram exec sessions e o tratamento formal de órfãos gerados por `setns/exec`. Só depois disso vale investir mais fundo em refinamentos como suporte avançado a job control, remapeamentos específicos de exit code, rootless e recuperação robusta de reconnect.

Esse roadmap é intencionalmente conservador. Em runtime de containers, obter o comportamento simples e correto vale mais do que acumular features cedo. Um init pequeno e previsível é melhor do que um init “esperto” que mistura signal proxy, parsing de shell, convenções de CLI e lógica de policy em um único binário opaco.

9.2 Casos de teste que realmente validam a feature

Testes unitários são úteis para normalização de estados e mapeamentos de erro, mas a essência dessa feature só aparece em integração com processos reais. É indispensável ter um conjunto de testes de sistema que faça o seguinte. Um workload que sai com código 3 deve fazer o container terminar com 3. Um workload inexistente deve falhar no handshake de start e produzir 127. Um script shell que cria child em background e ignora SIGTERM deve se comportar de forma diferente conforme a política de forwarding seja *Process* ou *ProcessGroup*. Um processo que double-forka deve gerar órfão reparentado ao PID 1 e não deixar zombie. Uma sessão de exec deve poder morrer sem alterar o status do container principal. Um timeout de stop deve escalar para hard kill e registrar a causa administrativa. E, principalmente, o container não deve encerrar o namespace cedo demais quando o workload sai antes de sobras serem drenadas.

Cenário	Comportamento esperado	Falha que esse teste detecta
Workload simples exit 3	Status final do container = 3.	Confusão entre código do wrapper e do workload.
Binário inexistente	<i>Start</i> falha com pre-exec estruturado; CLI projeta 127.	Container marcado como running sem execve real.
Shell com child em background	Em <i>ProcessGroup</i> , both shell e child recebem o sinal; em <i>Process</i> , apenas shell.	Signal forwarding implícito ou inconsistente.
Double-fork	Órfão é adotado pelo PID 1 e reaped corretamente.	Zombie leak e ausência de reaping de órfãos.
Exec session morre depois do primário	Container preserva status do primário; exec recebe status próprio.	Exec sobrescrevendo resultado oficial.
Stop com timeout expirado	Supervisor escalona hard kill, registra <i>FORCED_KILL</i> ou projeção equivalente.	Containers presos indefinidamente ou status impreciso.
Primário sai mas sobram descendentes	PID 1 drena/remata sobras antes de encerrar o namespace.	Término prematuro do PID 1 e SIGKILL implícito do namespace.

9.3 Hardening e recovery operacional

Depois de a semântica central estar correta, o próximo investimento deve ser recovery e observabilidade. O daemon C++ precisa ser capaz de listar tasks vivas, reconectar ao control plane Python, reemitir estado observado e manter consistência mesmo se houve queda do processo de API. O supervisor deve publicar eventos suficientemente ricos para que o Python reconstrua o histórico sem adivinhar. Em ambientes reais, a metade do valor dessa feature está na previsibilidade em incidentes: saber qual sinal foi enviado, se foi forwardado para grupo, quantos órfãos foram coletados, se houve escalada e qual foi a causa final do término.

Vale, ainda, rodar testes de caos direcionados. Mate o processo Python no meio de um stop; reinicie o daemon de API durante um workload longo; force criação de vários children e exec sessions; simule atraso entre registro de status e término do PID 1. O objetivo não é ser dramático, mas expor cedo os pontos onde desired state, observed state e kernel state podem divergir. Um runtime maduro é aquele que continua explicável quando algo dá errado.

10. Conclusão

Implementar corretamente a feature “PID namespace init” é, na prática, decidir se o seu projeto será um runtime de containers sério ou apenas um launcher que aproveita namespaces. O kernel Linux já definiu as regras do jogo: o primeiro processo do PID namespace é especial, precisa poder receber sinais de operação, assume órfãos, deve fazer reaping e, se morrer cedo, derruba o restante do namespace [1][2][3]. O Docker, Tini, dumb-init e containerd passaram anos ensinando as consequências operacionais disso [11][15][16][17]. O melhor caminho para um projeto novo é absorver essas lições em um design nativo, não reproduzir defeitos históricos por inércia.

A recomendação final desta pesquisa é clara. Mantenha Python como cérebro de orquestração, metadados e API. Use Cython como uma fronteira tipada e estreita. Coloque em C++ todo o mecanismo que fala com o kernel. Modele um supervisor host-side que usa `pidfd` para lifecycle confiável. Injetar um *runtime-init* interno e mínimo deve ser a estratégia principal para ambientes gerenciados, com um perfil compatível para quem realmente precisar de aderência ao comportamento clássico do Docker. Preserve o exit status do workload como primeiro cidadão da modelagem. E trate órfãos, zombies, signals e shutdown como partes da mesma responsabilidade, não como tópicos desconectados.

Se esse plano for seguido, o resultado não será apenas uma feature isolada bem implementada. Será um alicerce correto para todo o restante do runtime: exec, TTY, observabilidade, retry, recuperação após crash, integração com scheduler e, no futuro, políticas mais avançadas de segurança e isolamento. O ponto crítico é disciplinar a arquitetura desde já. Quando a semântica de processo está certa, muita coisa acima dela fica simples. Quando ela nasce errada, o restante do sistema passa anos compensando com remendos.

11. Referências

Todas as referências abaixo foram consultadas em 12 de março de 2026. Priorizou-se documentação oficial de kernel, Docker, OCI, containerd e projetos de init leve amplamente usados no ecossistema.

- [1] Michael Kerrisk, "pid_namespaces(7) - Linux manual page". man7.org. https://man7.org/linux/man-pages/man7/pid_namespaces.7.html

- [2] Michael Kerrisk, "kill(2) - Linux manual page". man7.org. <https://man7.org/linux/man-pages/man2/kill.2.html>

- [3] Michael Kerrisk, "wait(2) - Linux manual page". man7.org. <https://man7.org/linux/man-pages/man2/wait.2.html>

- [4] Michael Kerrisk, "clone(2) - Linux manual page". man7.org. <https://man7.org/linux/man-pages/man2/clone.2.html>

- [5] Michael Kerrisk, "pidfd_open(2) - Linux manual page". man7.org. https://man7.org/linux/man-pages/man2/pidfd_open.2.html

- [6] Michael Kerrisk, "pidfd_send_signal(2) - Linux manual page". man7.org. https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html

- [7] Michael Kerrisk, "signalfd(2) - Linux manual page". man7.org. <https://man7.org/linux/man-pages/man2/signalfd.2.html>

- [8] Michael Kerrisk, "PR_SET_CHILD_SUBREAPER(2const) - Linux manual page". man7.org. https://man7.org/linux/man-pages/man2/PR_SET_CHILD_SUBREAPER.2const.html

- [9] Michael Kerrisk, "PR_SET_PDEATHSIG(2const) - Linux manual page". man7.org. https://man7.org/linux/man-pages/man2/PR_SET_PDEATHSIG.2const.html

- [10] Docker Engine documentation, "Running containers". <https://docs.docker.com/engine/containers/run/>

- [11] Docker CLI reference, "docker container run". <https://docs.docker.com/reference/cli/docker/container/run/>

- [12] Docker CLI reference, "docker container stop". <https://docs.docker.com/reference/cli/docker/container/stop/>

- [13] Docker CLI reference, "docker container attach". <https://docs.docker.com/reference/cli/docker/container/attach/>

- [14] Dockerfile reference, "STOPSIGNAL". <https://docs.docker.com/reference/dockerfile/#stopsignal>

- [15] Tini README. <https://github.com/krallin/tini>

- [16] dumb-init README. <https://github.com/Yelp/dumb-init>

-
- [17]** containerd Runtime v2 README. <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md>
-
- [18]** OCI Runtime Specification, runtime.md. <https://github.com/opencontainers/runtime-spec/blob/main/runtime.md>
-
- [19]** Michael Kerrisk, "simple_init.c" (TLPI example). https://man7.org/tlpi/code/online/book/namespaces/simple_init.c.html
-